Java Programming

These slides contain either partial or complete content from various websites, presentations and other internet based resources. The author acknowledges the contributions of all those whose work has become part of these slides either in part or full and assures that these have been used for padagogical purposes only.

Index

- Hierarchical abstractions Base class object.
- subclass, subtype, substitutability.
- Forms of inheritance- specialization, construction, extension, limitation, combination.
- Benefits of inheritance, costs of inheritance.
- Member access rules, super uses, using final with inheritance.

Index

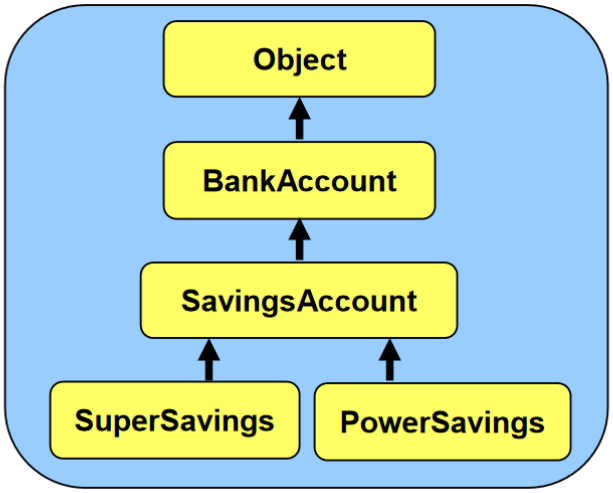
- Polymorphism- method overriding, abstractclasses.
- Defining, Creating and Accessing a Package
- Importing packages
- Differences between classes and interfaces
- Defining an interface
- Implementing interface
- Applying interfaces
- Variables in interface and extending interfaces

Inheritance

- Allows a software developer to reuse a sequence of statements
- Inheritance allows a software developer to reuse classes by deriving a new class from an existing one
- The existing class is called the parent class, or superclass, or base class
- The derived class is called the child class or subclass.
- As the name implies, the child inherits characteristics of the parent That is, the child class inherits the methods and data defined for the parent class

Inheritance

Inheritance relationships are often shown graphically in a class diagram, with the arrow pointing to the parent class



Inheritance should create an *is-a relationship*, meaning the child *is a* more specific version of the parent

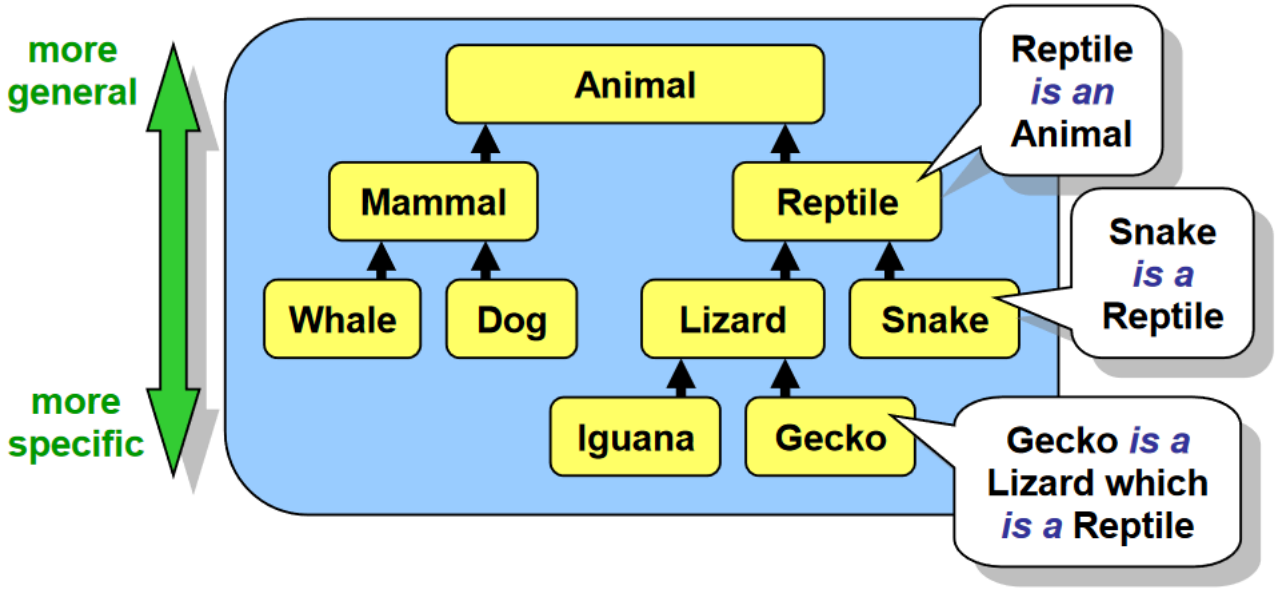
Deriving Subclasses

In Java, we use the reserved word extends to establish an inheritance relationship

```
class Animal
{
// class contents
int weight;
public void int getWeight() {…}
}
class Bird extends Animal
{
// class contents
public void fly() {…};
}
```

Class Hierarchy

A child class of one parent can be the parent of another child, forming class hierarchies



★At the top of the hierarchy there's a default class called Object.

Class Hierarchy

- Good class design puts all common features as high in the hierarchy as reasonable
- Inheritance is transitive
  - An instance of class Iguana is also an instance of Lizard, an instance of Reptile, …, and an instance of class Object
- The class hierarchy determines how methods are executed:
  - Previously, we took the simplified view that when variable v is an instance of class L, then a procedure call v.proc1() invokes the method proc1() defined in class L
  - However, if L is a child of some superclass L' (and hence v is both an instance of L and an instance of L'), the picture becomes more complex, because methods of class L can override the methods of class L' .

Defining Methods in the Child Class:

Overriding by Replacement

- A child class can override the definition of an inherited method in favor of its own
  - that is, a child can redefine a method that it inherits from its parent
  - the new method must have the same signature as the parent's method, but can have different code in the body
- In java, all methods except constructors override the methods of their ancestor class by replacement. E.g.:
  - the Animal class has method eat()
  - the Bird class has method eat() and Bird extends Animal
  - variable b is of class Bird, i.e. Bird b = …
  - b.eat() simply invokes the eat() method of the Bird class
- If a method is declared with the final modifier, it cannot be overridden

Defining Methods in the Child Class:

Overriding by Refinement

- Constructors in a subclass override the definition of an inherited constructor method by refining them (instead of replacing them)

– Assume class Person has constructors

Person(), Person(int weight), Person(int weight, int height)

– Assume class Student which extends Person has constructors

Student(), Student(int weight), Student(int weight, int height)

– Let's say we create a Student object, e.g. Student s = Student(50)

– This will invoke first the constructor of the Person (the superclass of Student) and then the constructor of the Student

Defining Methods in the Child Class:

This is called constructor chaining: If class C0 extends C1 and C1 extends C2
and … Cn-1 extends Cn = Object then when creating an instance of object C0
first constructor of Cn is invoked, then constructors of Cn-1, …, C2, C1, and
finally the constructorof C

- The constructors (in each case) are chosen by their signature, e.g. (), (int), etc…
- If no constructor with matching signature is found in any of the class Ci for i>0 then the default constructor is executed for that class
- If no constructor with matching signature is found in the class C0 then this causes a compiler error

First the new method must have the same signature as the parent's method, but can have different code in the body

Hierarchical Abstraction

- An essential element of object-oriented programming is abstraction.
- Humans manage complexity through abstraction. For example, people do not think of a vehicle as a set of tens of thousands of individual parts. They think of it as a well-defined object with its own unique behavior.
- This abstraction allows people to use a vehicle without being overwhelmed by the complexity of the parts that form the vehicle. They can ignore the details of how the engine, transmission, and braking systems work.
- Instead they are free to utilize the object as a whole.

Base Class Object

- In Java, all classes use inheritance.
- If no parent class is specified explicitly, the base class Object is implicitly inherited.
- All classes defined in Java, are a child of Object class, which provides minimal functionality common to all objects.
- Methods defined in Object class are;

1 equals(Object obj): Determine whether the argument object is the same as the receiver.

2 getClass(): Returns the class of the receiver, an object of type Class.

3 hashCode(): Returns a hash value for this object. Should be overridden when the equals method is changed.

4 toString(): Converts object into a string value. This method is also often overridden.

Base Class Object

Base class

1) a class obtains variables and methods from another class
2) the former is called subclass, the latter super-class (Base class)
3) a sub-class provides a specialized behavior with respect to its super-class
4) inheritance facilitates code reuse and avoids duplication of data

Extends

1) Is a keyword used to inherit a class from another class
   Allows extending only from one class

Subclass, Subtype and Substitutability

- A subtype is a class that satisfies the principle of substitutability.
- A subclass is something constructed using inheritance, whether or not it satisfies the principle of substitutability.
- The two concepts are independent. Not all subclasses are subtypes, and (at least in some languages) you can construct subtypes that are not subclasses.
- Substitutability is fundamental to many of the powerful software development techniques in OOP.
- Substitutability can occur through use of inheritance, whether using extends, or using implements keywords.

Subclass, Subtype and Substitutability

When new classes are constructed using inheritance, the argument used to justify the validity of substitutability is as follows;

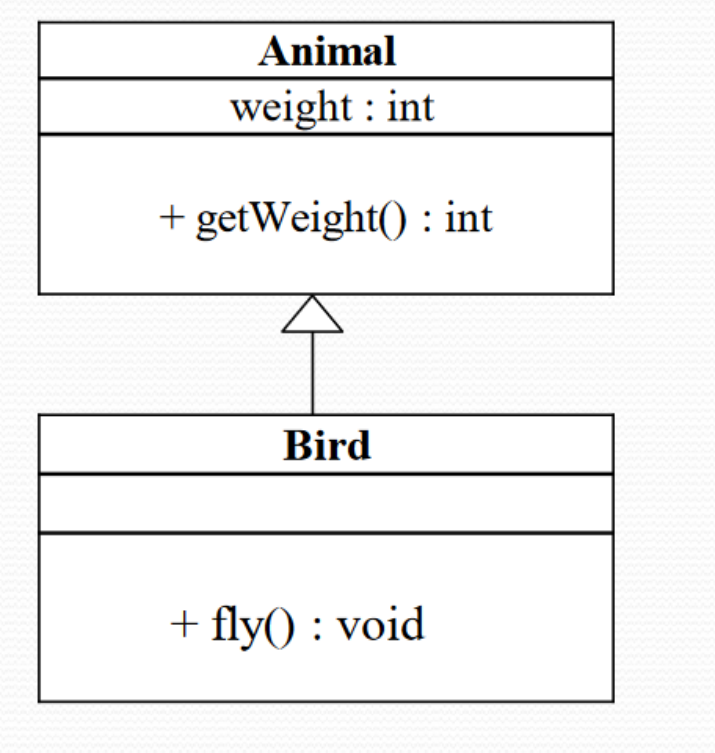
- Instances of the subclass must possess all data fields associated with its parent class.
- Instances of the subclass must implement, through inheritance at least, all functionality defined for parent class. (Defining new methods is not important for the argument.)
- Thus, an instance of a child class can mimic the behavior of the parent class and should be indistinguishable from an instance of parent class if substituted in a similar situation.

Subclass, Subtype and Substitutability

- The term subtype is used to describe the relationship between types that explicitly recognize the principle of substitution. A type ‘B’ is considered to be a subtype of ‘A’ if an instance of ‘B’ can legally be assigned to a variable declared as of type ‘A’.
- The term subclass refers to inheritance mechanism made by extends keyword.
- Not all subclasses are subtypes. Subtypes can also be formed using interface, linking types that have no inheritance relationship

Subtype

Inheritance relationships are often shown graphically in a class diagram, with the arrow pointing to the parent class



Substitutability (Deriving Subclasses)

In Java, we use the reserved word extends to establish an inheritance relationship

```
class Animal
{
// class contents
int weight;
public void int getWeight() {…}
}
class Bird extends Animal
{
// class contents
public void fly() {…};
}
```

Forms of Inheritance

Inheritance is used in a variety of way and for a variety of different purposes

- Inheritance for Specialization
- Inheritance for Specification
- Inheritance for Construction
- Inheritance for Extension
- Inheritance for Limitation
- Inheritance for Combination

One or many of these forms may occur in a single case

Forms of Inheritance - Specialization

By far the most common form of inheritance is for specialization.

- Child class is a specialized form of parent class
- Principle of substitutability holds

A good example is the Java hierarchy of Graphical components in the AWT:

- Component
  - Label
  - Button
  - TextComponent
    - TextArea
    - TextField
  - CheckBox
  - ScrollBar

Forms of Inheritance - Specification

The next most common form of inheritance involves specification. The parent class specifies some behavior, but does not implement the behavior

Child class implements the behavior

Similar to Java interface or abstract class

When parent class does not implement actual behavior but merely defines the behavior that will be implemented in child classes

Example, Java 1.1 Event Listeners: ActionListener, MouseListener, and so on specify behaviour, but must be subclassed.

Forms of Inheritance- Construction

Child class inherits most of its functionality from parent, but may change the name or parameters of methods inherited from parent class to form its interface.

This type of inheritance is also widely used for code reuse purposes. It simplifies the construction of newly formed abstraction but is not a form of subtype, and often violates substitutability.

Example is Stack class defined in Java libraries.

Forms of Inheritance - Extension

Subclassification for extension occurs when a child class only adds new behavior to the parent class and does not modify or alter any of the inherited attributes.

Such subclasses are always subtypes, and substitutability can be used.

Example of this type of inheritance is done in the definition of the class Properties which is an extension of the class HashTable.

Generalization -- Extension

- The child class generalizes or extends the parent class by providing more functionality
- In some sense, opposite of subclassing for specialization
- The child doesn't change anything inherited from the parent, it simply adds new features
- Often used when we cannot modify existing base parent class

Example, ColoredWindow inheriting from Window

- Add additional data fields
- Override window display methods

Forms of Inheritance - Limitation

The child class limits some of the behavior of the parent class.

Example, you have an existing List data type, and you want a Stack

- Inherit from List, but override the methods that allow access to elements other than top so as to produce errors.

- Is not a subtype, and substitutability is not proper.

Forms of Inheritance - Combination

This type of inheritance is known as multiple inheritance in Object Oriented Programming.

Although Java does not permit a subclass to be formed by inheritance from more than one parent class, several approximations to the concept are possible.

Example of this type is ;

Mouse and TouchPad and JoyStick

Better solution, abstract out common parts to new parent class, and use subclassing for specialization.

Summary of Forms of Inheritance

- Specialization. The child class is a special case of the parent class; in other words, the child class is a subtype of the parent class.
- Specification. The parent class defines behavior that is implemented in the child class but not in the parent class.
- Construction. The child class makes use of the behavior provided by the parent class, but is not a subtype of the parent class.
- Generalization. The child class modifies or overrides some of the methods of the parent class.

Summary of Forms of Inheritance

- Extension. The child class adds new functionality to the parent class, but does not change any inherited behavior.
- Limitation. The child class restricts the use of some of the behavior inherited from the parent class.
- Variance. The child class and parent class are variants of each other, and the class-subclass relationship is arbitrary.
- Combination. The child class inherits features from more than one parent class. This is multiple inheritance and will be the subject of a later chapter.

The Benefits of Inheritance

- Software Reusability (among projects)
- Increased Reliability (resulting from reuse and sharing of well-tested code)
- Code Sharing (within a project)
- Consistency of Interface (among related objects)
- Rapid Prototyping (quickly assemble from pre-existing components)
- Polymorphism and Frameworks (high-level reusable components)
- Information Hiding

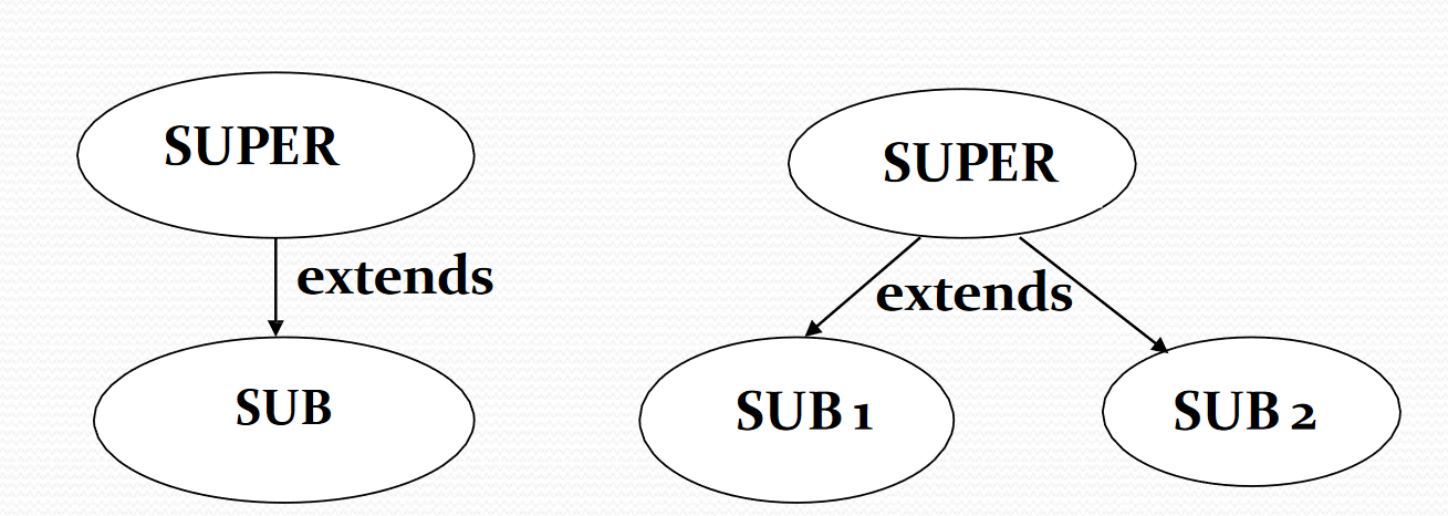
The Costs of Inheritance

- Execution Speed
- Program Size
- Message-Passing Overhead
- Program Complexity (in overuse of inheritance)

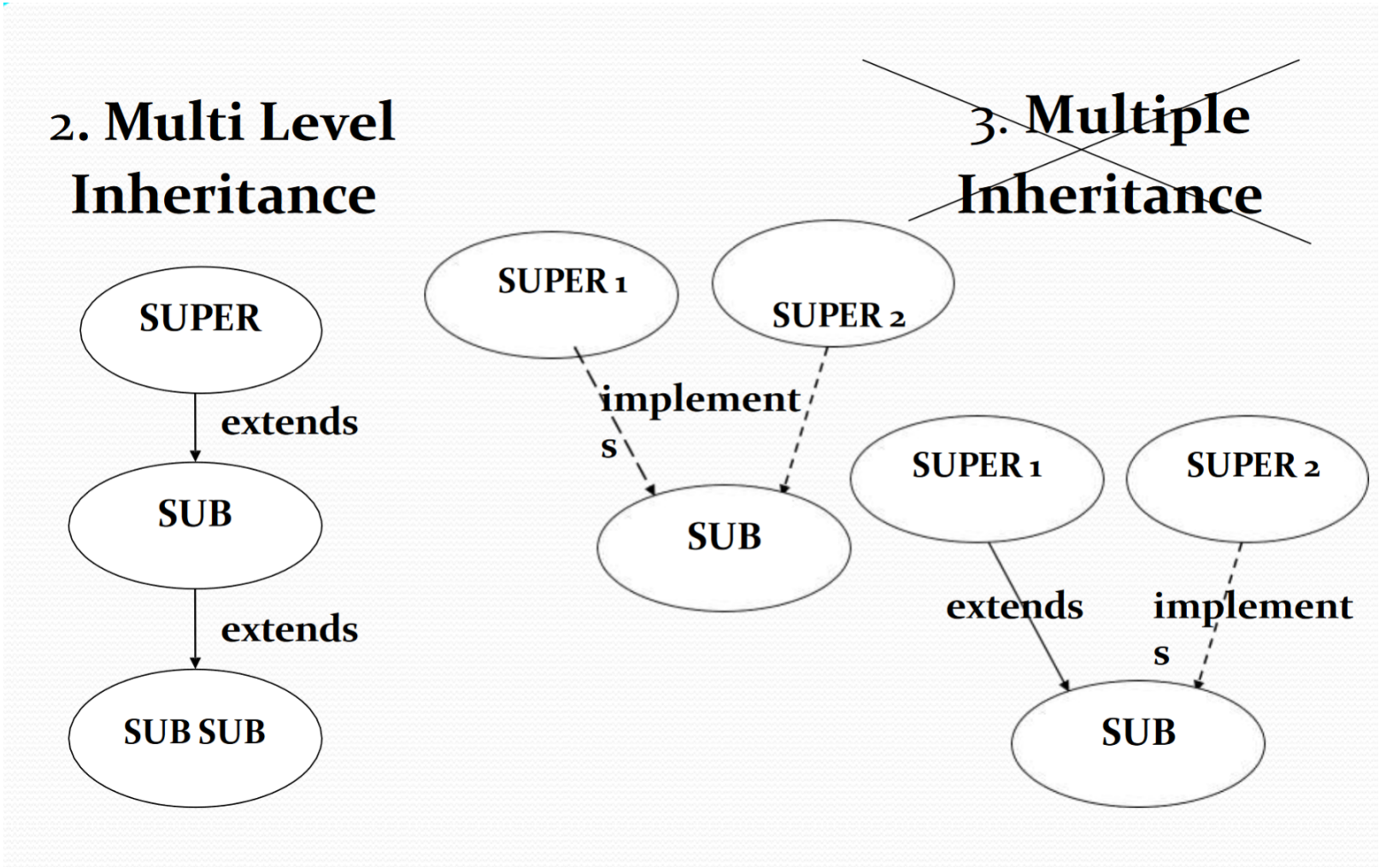
Types of inheritance

Acquiring the properties of an existing Object into newly creating Object to overcome the re-declaration of properties in deferent classes.

These are 3 types:



Types of inheritance



Member access rules

- Visibility modifiers determine which class members are accessible and which do not
- Members (variables and methods) declared with public visibility are accessible, and those with private visibility are not
- Problem: How to make class/instance variables visible only to its subclasses?
- Solution: Java provides a third visibility modifier that helps in inheritance situations: protected

Modifiers and Inheritance (cont.)

Visibility Modifiers for class/interface:

public : can be accessed from outside the class definition.

protected : can be accessed only within the class definition in which it appears, within other classess in the same package, or within the definition of subclassess.

private : can be accessed only within the class definition in which it appears.

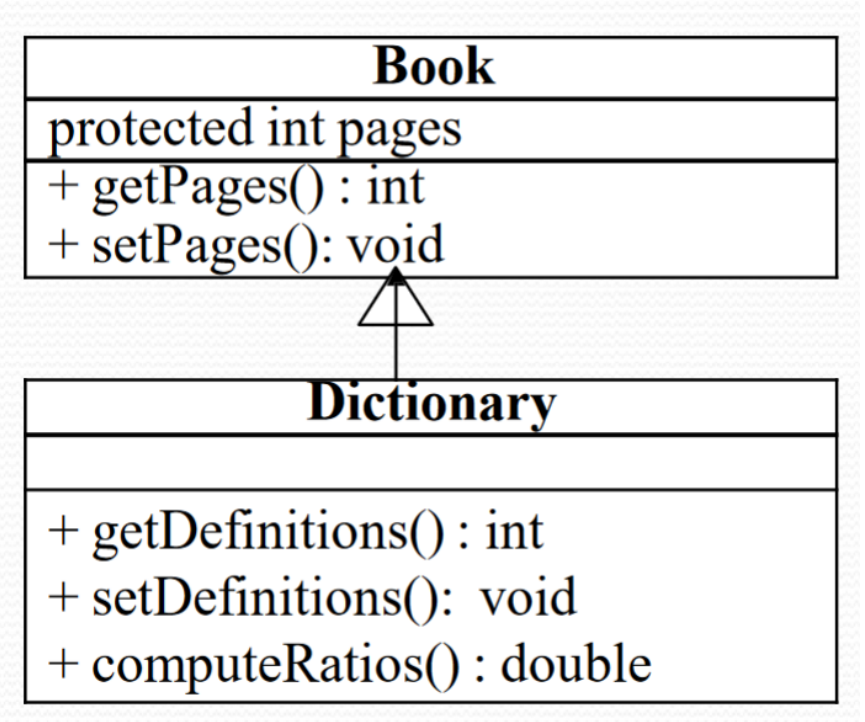
default-access (if omitted) features accessible from inside the current Java package

The protected Modifier

The protected visibility modifier allows a member of a base class to be accessed in the child

protected visibility provides more encapsulation than public does

protected visibility is not as tightly encapsulated as private visibility



“super” Keyword

‘super’ is a keyword used to refer to hidden variables of super class from sub class.

```
super.a=a;
```

It is used to call a constructor of super class from constructor of sub class which should be first statement.

```
super(a,b);
```

It is used to call a super class method from sub class method to avoid redundancy of code

```
super.addNumbers(a, b);
```

Super and Hiding

Why is super needed to access super-class members?

When a sub-class declares the variables or methods with the same names and types as its super-class:

```
class A {
int i = 1;
}

class B extends A {
int i = 2;
System.out.println(“i is “ + i);
}
```

The re-declared variables/methods hide those of the super-class.

Example: Super and Hiding

```
class A {
    int i;
}
class B extends A {
    int i;
    B(int a, int b) {
    super.i = a; i = b;
}
    void show() {
    System.out.println("i in superclass: " + super.i);
    System.out.println("i in subclass: " + i);
    }
}
```

Example: Super and Hiding

Although the i variable in B hides the i variable in A, super allows access to the hidden variable of the super-class:

```
class UseSuper {
    public static void main(String args[]) {
    B subOb = new B(1, 2);
    subOb.show();
    }
}
```

Using final with inheritance

final keyword is used to declare constants which cannot change its value of definition.

final Variables cannot change its value.

final Methods cannot be Overridden or Over Loaded

final Classes cannot be extended or inherited

Preventing Overriding with final

A method declared final cannot be overridden in any sub-class:

```
class A {
    final void meth() {
    System.out.println("This is a final method.");
    }
}
```

This class declaration is illegal:

```
class B extends A {
    void meth() {
    System.out.println("Illegal!");
    }
}
```

Preventing Inheritance with final

A class declared final cannot be inherited – has no sub-classes.

final class A { … }

This class declaration is considered illegal:

class B extends A { … }

Declaring a class final implicitly declares all its methods final.

It is illegal to declare a class as both abstract and final.

Polymorphism

Polymorphism is the ability of different objects to respond to the same message in different ways. This means that different objects can have very different method implementations for the same message.

Polymorphism is the ability of a new object to implement the base functionality of a parent object in a new way.

Polymorphism is an object's ability to behave differently depending on its type.

Polymorphism is the ability of objects belonging to different types to respond to methods of the same name, each one according to the appropriate type-specific behavior.

Polymorphism

One of the three pillars of object-orientation is Polymorphism.

Polymorphism: many different (poly) forms of objects that share a common interface respond differently when a method of that interface is invoked:

1) a super-class defines the common interface

2) sub-classes have to follow this interface (inheritance), but are also permitted to provide their own implementations (overriding)

A sub-class provides a specialized behaviors relying on the common elements defined by its super-class.

Method Overloading

- ***Method Overloading*** is the process of declaring methods with the same name but different parameter types.

- A method can be overloaded in the same class or in a subclass.

- Which overloaded method to call is based on *reference type* and decided at *compile time.*

- Which overridden method to call is based on *object type* and decided during *runtime.*

Rules of Method Overloading

1. Overloaded methods *must* change the argument list.

2. Overloaded methods can change the return type.

3. Overloaded methods can change the access modifier.

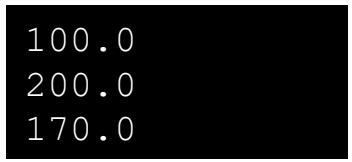
4. Overloaded methods can declare new or broader checked exceptions.

Implementing Method Overloading

```
public static void main(String[] args) {
    Sales s = new Sales();

    System.out.println(s.computeSales(100));
    System.out.println(s.computeSales(100,2));
    System.out.println(s.computeSales(100,2,30));
}

class Sales {
    double computeSales(double price) {
           double sales;
           sales = price;
           return sales;
    }
    double computeSales(double price, int qty) {
           double sales;
           sales = price * qty;
           return sales;
    }
    double computeSales(double price, int qty, double discount) {
           double sales;
           sales = (price * qty) - discount;
           return sales;
    }
}
```



Method Overriding

- ***Method Overriding*** allows a subclass to redefine methods of the same name from the superclass.
- The key benefit of overriding is the ability to define/defer behavior specific to subclasses.
- Which overridden method to call is based on *object type* and decided at *runtime.*

Rules of Method Overriding

1. *An overridden method must* have
   - the same name
   - the same number of parameters and types
   - the same return type

   as the *overridden method.*
2. Overriding a method cannot narrow the method access level defined in the overridden method.
3. Overriding a method cannot widen checked exceptions defined in the overridden method.
4. Methods declared as `private`, `static`, or `final` cannot be overridden.
5. A static method cannot override an instance method.

Implementing Method Overriding

```
public static void main(String[] args) {
    Sales s = new Sales();
    Sales st = new SalesTax();

    System.out.println(s.computeSales(100));
    System.out.println(s.computeSales(100,2));
    System.out.println(s.computeSales(100,2,30));
    System.out.println(st.computeSales(100));
}
class Sales {
    double computeSales(double price) {
          double sales = price;
          return sales;
    }
    double computeSales(double price, int qty) {
          double sales = price * qty;
          return sales;
    }
    double computeSales(double price, int qty, double discount)
          double sales = (price * qty) - discount;
          return sales;
    }
}

class SalesTax extends Sales {
    double computeSales(double price) {
          double sales = price * 1.10; // add tax
          return sales;
    }
}
```

Define *Sales* reference variable containing *SalesTax* object

It determined which method to run based on *object type* (SalesTax) instead of *reference type* (Sales)

```
100.0
200.0
170.0
110.0
```

Overloading vs Overriding

Criteria	***Overloaded Method***	***Overridden Method***
Argument list	Different	Same
Return type	Can change	Same
Exceptions	Can change	Cannot be wider
Access level	Can change	Cannot be narrower
Invocation	Based on reference type and decided at compile time	Based on object type and decided at runtime

Early Binding and Late Binding

- ***Early binding*** means translating operations or associating identifiers during compile time.

- ***Late binding*** means delaying translation of an operation or associating identifiers at runtime (also known as *dynamic method lookup* or *virtual method invocation*). It is used in polymorphism to determine the actual method invoked (depending on the type of the actual object).

Implementing Polymorphism

```
class Animal {
    public void eat() { System.out.println("Animal eating...");}
}
class Snake extends Animal {
    public void eat() { System.out.println("Snake eating...");}
}
class Horse extends Animal {
    public void eat() { System.out.println("Horse eating...");}
    public void eat(String s) { System.out.println("Horse eating " + s);    }
}

public static void main(String[] args) {
    Animal pig = new Animal();
    Snake viper = new Snake();
    Horse stallion = new Horse();
    Animal animalViper = new Snake();
    Animal animalStallion = new Horse();

    pig.eat();                  // will this compile? Output
    pig.eat("left overs");      //will this compile? Output
    viper.eat();                // will this compile? Outpu
    viper.eat("chicken");       //will this compile? Output
    stallion.eat();             // will this compile? Output?
    stallion.eat("grass");      // will this compile? Output?
    animalViper.eat();          // will this compile? Output?
    animalViper.eat("rat");     //will this compile? Output?
    animalStallion.eat();       // will this compile? Output?
    animalStallion.eat("Carrots");  //will this compile? Output?
}
```

```
Animal eating...
Snake eating...
Horse eating...
Horse eating grass
Snake eating...
Horse eating...
```

Key Points

- *Polymorphism* is an object's ability to behave differently depending on its type
- *Method Overloading* is the process of declaring methods with the same name but different parameter types
- *Method Overriding* allows a subclass to redefine methods of the same name from a superclass
- Early binding translates identifiers at compile-time while late binding translates identifiers during runtime
- Late binding is used in polymorphism to determine the actual method invoked at runtime

Polymorphism

A polymorphic reference can refer to different types of objects at different times

In java every reference can be polymorphic except references to base types and final classes.

It is the type of the object being referenced, not the reference type, that determines which method is invoked

Polymorphic references are therefore resolved at run-time, not during compilation; this is called dynamic binding

Careful use of polymorphic references can lead to elegant, robust software designs

Interfaces

- A Java *interface* is a collection of abstract methods and constants
- An *abstract method* is a method header without a method body
- An abstract method can be declared using the modifier `abstract`, but because all methods in an interface are abstract, usually it is left off
- An interface is used to establish a set of methods that a class will implement

Interfaces

interface is a reserved word



None of the methods in an interface are given a definition (body)

```
public interface Doable
{
    public void doThis();
    public int doThat();
    public void doThis2 (float value, char ch);
    public boolean doTheOther (int num);
}
```



A semicolon immediately follows each method header

Interfaces

- An interface cannot be instantiated
- Methods in an interface have public visibility by default. The fields in an interface are implicitly public static final.
- A class formally implements an interface by:
  - stating so in the class header
  - providing implementations for each abstract method in the interface
- If a class asserts that it implements an interface, it must define all methods in the interface

Interfaces

```
public class CanDo implements Doable
{
    public void doThis ()
    {
        // whatever
    }

    public void doThat ()
    {
        // whatever
    }

    // etc.
}
```

implements is a reserved word

Each method listed in **Doable** is given a definition

Interfaces

- A class that implements an interface can implement other methods as well
- In addition to (or instead of) abstract methods, an interface can contain constants
- When a class implements an interface, it gains access to all its constants

Comparison with Inheritance

- Interfaces don't define any method actions… you need to fill in all the details
- It essentially just gives a basic collection of method names
- Not a strict hierarchy
- Important: can implement several Interfaces
- Can use this to "fake" multiple inheritance

Interfaces

- A class can implement multiple interfaces
- The class must implement all methods in all interfaces listed in the header

```
class ManyThings implements
  interface1, interface2
{
    // all methods of both interfaces
}
```

Built-in Interfaces

- The Java standard library includes lots more built-in interfaces
  - they are listed in the API with the classes
- Examples:
  - `Clonable` – implements a `clone()` method
  - `Formattable` – can be formatted with `printf`

Implementing versus Inheriting

- Implementing an Interface is very similar to inheriting a class

```
class MyClass implements MyInterface {…}
```

  - Takes everything from MyInterface and puts it in MyClass
  - Except **all** the methods must be implemented here
  - No previous implementations to fall back on

What is an Abstract Class?

Definition:
An abstract class is a class that cannot be instantiated on its own and is meant to be subclassed. It can contain abstract methods (methods without a body) that must be implemented by subclasses, as well as concrete methods (methods with a body) that can be inherited.

Characteristics:
Abstract Methods: Methods without an implementation; must be implemented by subclasses.
Concrete Methods: Methods with an implementation; can be inherited or overridden by subclasses.
Constructors: Can have constructors which can be called by subclass constructors.
Fields: Can have fields (variables), including static fields.

When to Use Abstract Classes:

- When you need to provide common base functionality that multiple subclasses can share.
- When you want to prevent direct instantiation and enforce certain methods to be implemented by subclasses.
- When you have a base class that should provide some default behavior.

When to Use Interfaces:

- When you want to define a contract that multiple classes can implement, regardless of their class hierarchy.
- When you need to support multiple inheritance of behavior.

Interfaces vs. Abstract Classes

- Similarities
  - neither can be instantiated
  - both can be used as the starting point for a class
- Differences
  - A class can contain implementations of methods
  - A class can implement many interfaces, but only one class

Comparison

- In order of “abstractness”:
  - Interface
    - no method implementations
    - can’t be instantiated
  - Abstract class
    - some method implementations
    - can’t be instantiated
  - Non-abstract class
    - all methods implemented
    - can be instantiated

Abstract Method

Inheritanceallows a sub-class to overridethe methods of its super-class.

A super-class may altogether leave the implementation details of a method and declaresuch a method abstract:

abstract type name(parameter-list);

Two kinds of methods:

1) concrete – may be overridden by sub-classes

2) abstract – must be overridden by sub-classes

It is illegal to define abstract constructorsor static methods.

Defining a Package

A package is both a naming and a visibility control mechanism:

1)divides the name space into disjoint subsets. It is possible to define classes within a package that are not accessible by code outside the package.

2)controls the visibility of classes and their members It is possible to define class members that are only exposed to other members of the same package.

Same-package classes may have an intimate knowledge of each other, but not expose that knowledge to other packages

Creating a Package

A package statement inserted as the first line of the source file:

```
package myPackage;

class MyClass1 { … }

class MyClass2 { … }
```

means that all classes in this file belong to the myPackage package.

The package statement creates a name space where such classes are stored.

When the package statement is omitted, class names are put into the default package which has no name.

Multiple Source Files

Other files may include the same packageinstruction:

1. package myPackage;

class MyClass1 { … }

class MyClass2 { … }

2. package myPackage;

class MyClass3{ … }

A package may be distributed through several source files

Packages and Directories

Java uses file system directories to store packages.

Consider the Java source file:

```
package myPackage;

class MyClass1 { … }

class MyClass2 { … }
```

The byte code files MyClass1.class and MyClass2.class must be stored in a directory myPackage.

Case is significant! Directory names must match package names exactly.

Package Hierarchy

To create a package hierarchy, separate each package name with a dot:

```
package myPackage1.myPackage2.myPackage3;
```

A package hierarchy must be stored accordingly in the file system:

1) Unix myPackage1/myPackage2/myPackage3

2) Windows myPackage1\myPackage2\myPackage3

3) Macintosh myPackage1:myPackage2:myPackage3

You cannot rename a package without renaming its directory!

Accessing a Package

As packages are stored in directories, how does the Java run-time system know where to look for packages?

Two ways:

1)The current directory is the default start point - if packages are stored in the current directory or sub-directories, they will be found.

2)Specify a directory path or paths by setting the CLASSPATH environment variable.

CLASSPATH Variable

CLASSPATH - environment variable that points to the root directory of the system's package hierarchy.

Several root directories may be specified in CLASSPATH,

e.g. the current directory and the C:\raju\myJavadirectory:

.;C:\raju\myJava

Java will search for the required packages by looking up subsequent directories described in the CLASSPATH variable.

Finding Packages

Considerthis package statement:

package myPackage;

In order for a program to find myPackage, one of the following must be true:

1) program is executed from the directory immediately above myPackage (the parent of myPackagedirectory)

2) CLASSPATH must be set to include the path to myPackage

Example: Package

```
package MyPack;
class Balance {
    String name;
    double bal;
    Balance(String n, double b) {
        name = n; bal = b;
    }
    void show() {
        if (bal<0) System.out.print("-->> ");
        System.out.println(name + ": $" + bal);
    }
}
```

Example: Package

```
class AccountBalance
{
    public static void main(String args[])
    {
        Balance current[] = new Balance[3];
        current[0] = new Balance("K. J. Fielding",
        123.23);
        current[1] = new Balance("Will Tell", 157.02);
        current[2] = new Balance("Tom Jackson",
        -12.33);
        for (int i=0; i<3; i++) current[i].show();
    }
}
```

Example: Package

Save, compile and execute:

1) call the file AccountBalance.java

2) save the file in the directory MyPack

3) compile; AccountBalance.class should be also in MyPack

4)set access to MyPack in CLASSPATH variable, or make the parent of MyPack your current directory

5) run: java MyPack.AccountBalance

Make sure to use the package-qualified class name.

Importing of Packages

Since classes within packages must be fully-qualified with their package names, it would be tedious to always type long dot-separated names.

The import statement allows to use classes or whole packages directly.

Importing of a concrete class:

```
import myPackage1.myPackage2.myClass;
```

Importing of all classes within a package:

```
import myPackage1.myPackage2.*;
```

Import Statement

The import statement occurs immediatelyafter the Package statement and before the class statement:

```
package myPackage;
import otherPackage1;otherPackage2.otherClass;
class myClass { … }
```

The Java system accepts this import statement by default:

```
import java.lang.*;
```

This package includes the basic language functions. Without such functions, Java is of no much use.

Example: Packages 1

A package MyPack with one public class Balance. The class has two same-package variables: public constructor and a public show method.

```
package MyPack;
public class Balance {
	String name;
	double bal;
	public Balance(String n, double b) {
		name = n; bal = b;
	}
	public void show() {
		if (bal<0) System.out.print("-->> ");
		System.out.println(name + ": $" + bal);
	}
}
```

Example: Packages 2

The importing code has access to the public class Balance of the MyPack package and its two public members:

```
import MyPack.*;
class TestBalance {
    public static void main(String args[]) {
        Balance test = new Balance("J. J. Jaspers",
        99.88);
        test.show();
    }
}
```

Java Source File

Finally, a Java source file consists of:

1) a single package instruction (optional)

2) several import statements (optional)

3) a single public class declaration (required)

4) several classes private to the package (optional)

At the minimum, a file contains a single public class declaration.

THANK
YOU

Method Overriding

When a method of a sub-class has the same name and type as a method of the super-class, we say that this method is overridden.

When an overridden method is called from within the sub-class:

1) it will always refer to the sub-class method

2) super-class method is hidden

Example: Hiding with Overriding 1

```
class A {
    int i, j;
    A(int a, int b) {
    i = a; j = b;
    }
    void show() {
    System.out.println("i and j: " + i + " " + j);
    }
}
```

Example: Hiding with Overriding 2

```
class B extends A {
    int k;
    B(int a, int b, int c) {
        super(a, b);
        k = c;
    }
    void show() {
    System.out.println("k: " + k);
    }
}
```

Example: Hiding with Overriding 3

When show() is invoked on an object of type B, the version of show() defined in B is used:

```
class Override {
    public static void main(String args[]) {
    B subOb = new B(1, 2, 3);
    subOb.show();
    }
}
```

The version of show() in A is hidden through overriding.

Overloading vs. Overriding

- Overloading deals with multiple methods in the same class with the same name but different signatures
- Overriding deals with two methods, one in a parent class and one in a child class, that have the same signature
- Overloading lets you define a similar operation in different ways for different data
- Overriding lets you define a similar operation in different ways for different object types